Question 1

Campus Event Financial Planning

1. Problem Statement

A college student team is planning a small campus event. The program calculates venue, equipment, catering, transport and certificate costs. It then finds the amount contributed by participants, sponsorship shortfall, and the minimum number of registrations needed to break even.

2. Objectives

• Calculate total event cost.

• Calculate participant contribution.

• Find sponsorship shortfall.

• Calculate minimum registrations needed to break even.

• Display a simple financial summary.

3. Assumptions

• All costs are entered as simple numbers.

• Participant fee is fixed.

• Sponsorship amount is fixed.

• Break-even registrations = remaining cost / participant fee, rounded up.

• No advanced libraries are used.

4. Algorithm

• Start.

• Read event costs and participant details.

• Add all costs to get total cost.

• Calculate participant contribution.

• Find remaining amount after sponsorship.

• Calculate minimum registrations for break-even.

• Display results.

• Stop.

5. Flowchart

```
START → Input / Store Data → Calculate / Process → Check Conditions → Display Result → STOP
```

6. Pseudocode

```
START
INPUT venue, equipment, catering, transport, certificates
INPUT participants, fee, sponsorship
total = venue + equipment + catering + transport + certificates
contribution = participants * fee
remaining = total - sponsorship - contribution
IF remaining > 0
    break_even = CEILING((total - sponsorship) / fee)
ELSE
    break_even = 0
END IF
DISPLAY total, contribution, remaining, break_even
STOP
```

7. Python Source Code

```
venue = 5000
equipment = 2000
```

```
catering = 8000
transport = 1500
certificates = 500
participants = 40
fee = 300
sponsorship = 4000

total = venue + equipment + catering + transport + certificates
contribution = participants * fee
remaining = total - sponsorship - contribution

if total > sponsorship:
    break_even = (total - sponsorship + fee - 1) // fee
else:
    break_even = 0

print("Total event cost:", total)
print("Participant contribution:", contribution)
print("Sponsorship:", sponsorship)
print("Remaining amount:", remaining)
print("Minimum registrations to break even:", break_even)
```

8. Test Cases

Test / Item	Expected / Result
Total cost	17000
Participant contribution	12000
Sponsorship	4000
Remaining amount	1000
Break-even registrations	44

9. Sample Output

```
Total event cost: 17000
Participant contribution: 12000
Sponsorship: 4000
Remaining amount: 1000
Minimum registrations to break even: 44
```

10. Conclusion

This simple program uses variables, arithmetic operations and a basic condition to help a first-year student understand event budgeting and break-even calculation.

Question 2

Examination Result and Grace-Mark Validation

1. Problem Statement

A college wants to process marks in several subjects. The program checks subject-wise pass criteria, applies a small grace-mark policy when allowed, and produces the final result without allowing marks to exceed the maximum.

2. Objectives

• Store marks for subjects.

• Check pass marks.

• Apply grace marks where permitted.

• Keep marks within maximum marks.

• Display final result and classification.

3. Assumptions

• Each subject has a maximum of 100 marks.

• Pass mark is 40.

• At most 5 grace marks can be used for one subject.

• Grace marks are used only when the original mark is below 40 but not below 35.

• Final marks cannot exceed 100.

4. Algorithm

• Start.

• Store subject names and marks.

• Check each mark against pass criteria.

• If eligible, add grace marks up to the allowed limit.

• Check all subjects.

• Classify the result.

• Display final marks.

• Stop.

5. Flowchart

```
START → Input / Store Data → Calculate / Process → Check Conditions → Display Result → STOP
```

6. Pseudocode

```
START
marks = [72, 38, 65, 35]
FOR each mark
    IF mark >= 40
        keep mark
    ELSE IF mark >= 35
        mark = mark + 5
        IF mark > 100
            mark = 100
        END IF
    END IF
END FOR
IF every mark >= 40
    result = "PASS"
ELSE
    result = "FAIL"
END IF
DISPLAY marks and result
STOP
```

7. Python Source Code

```
subjects = ["Python", "Maths", "Web", "English"]
marks = [72, 38, 65, 35]

final_marks = []
for mark in marks:
    if mark >= 40:
        final_marks.append(mark)
    elif mark >= 35:
        mark = mark + 5
        if mark > 100:
            mark = 100
        final_marks.append(mark)
    else:
        final_marks.append(mark)

passed = True
for mark in final_marks:
    if mark < 40:
        passed = False

print("FINAL MARKS")
for i in range(len(subjects)):
    print(subjects[i], ":", final_marks[i])

if passed:
    print("Result: PASS")
else:
    print("Result: FAIL")
```

8. Test Cases

Test / Item	Expected / Result
Python	72
Maths	43
Web	65
English	40
Result	PASS

9. Sample Output

```
FINAL MARKS
Python : 72
Maths : 43
Web : 65
English : 40
Result: PASS
```

10. Conclusion

This program demonstrates lists, loops and if-else conditions. It is kept simple so a first-year student can understand how grace marks affect the final result.

Question 3

Attendance Recovery Computation

1. Problem Statement

A student has low attendance and wants to know how many consecutive classes must be attended to reach the required eligibility percentage. The program calculates the present attendance and the minimum future classes needed.

2. Objectives

• Calculate current attendance percentage.

• Find classes needed to reach the target.

• Show the result for the student.

3. Assumptions

• Current attendance is represented by attended and conducted classes.

• Target attendance is 75%.

• The student attends every future class considered.

• Classes are counted as whole numbers.

4. Algorithm

• Start.

• Read attended and conducted classes.

• Calculate current percentage.

• If already eligible, display 0 classes.

• Otherwise add one future attended class at a time until target is reached.

• Display required classes.

• Stop.

5. Flowchart

```
START → Input / Store Data → Calculate / Process → Check Conditions → Display Result → STOP
```

6. Pseudocode

```
START
INPUT attended, conducted
percentage = attended / conducted * 100
required = 0
WHILE attended / conducted * 100 < 75
    attended = attended + 1
    conducted = conducted + 1
    required = required + 1
END WHILE
DISPLAY percentage, required
STOP
```

7. Python Source Code

```
attended = 60
conducted = 90
target = 75
original = (attended / conducted) * 100
required = 0

while (attended / conducted) * 100 < target:
    attended += 1
    conducted += 1
    required += 1

print("Current attendance:", round(original, 2), "%")
```

```
print("Classes required:", required)
print("Attendance after recovery:", round(attended / conducted * 100, 2), "%")
```

8. Test Cases

Test / Item	Expected / Result
Current attendance	66.67%
Classes required	30
Recovered attendance	75.00%

9. Sample Output

```
Current attendance: 66.67 %
Classes required: 30
Attendance after recovery: 75.0 %
```

10. Conclusion

This program uses a while loop to simulate attending future classes. It helps a first-year student understand repeated calculation and percentage conditions.

Question 4

ATM Transaction Sequence Simulation

1. Problem Statement

This program simulates a simple ATM account. It processes deposits and withdrawals, checks whether withdrawals are valid, maintains the balance, and counts accepted and rejected transactions.

2. Objectives

• Maintain an account balance.

• Process deposits and withdrawals.

• Reject invalid withdrawals.

• Display transaction summary.

• Show final balance.

3. Assumptions

• Starting balance is 5000.

• Withdrawal cannot be more than the current balance.

• Each withdrawal must not exceed 10000.

• Deposits must be positive.

4. Algorithm

• Start with balance.

• Read each transaction.

• For deposit, add amount.

• For withdrawal, check limits and balance.

• Accept or reject transaction.

• Display final balance and summary.

• Stop.

5. Flowchart

```
START → Input / Store Data → Calculate / Process → Check Conditions → Display Result → STOP
```

6. Pseudocode

```
START
balance = 5000
FOR each transaction
    IF type = deposit
        balance = balance + amount
        accepted = accepted + 1
    ELSE IF type = withdrawal
        IF amount > 0 AND amount <= balance AND amount <= 10000
            balance = balance - amount
            accepted = accepted + 1
        ELSE
            rejected = rejected + 1
        END IF
    END IF
END FOR
DISPLAY balance, accepted, rejected
STOP
```

7. Python Source Code

```
balance = 5000
transactions = [("deposit", 2000), ("withdraw", 1500),
                ("withdraw", 7000), ("deposit", 1000),
```

```
                ("withdraw", 12000)]

accepted = 0
rejected = 0

for kind, amount in transactions:
    if kind == "deposit" and amount > 0:
        balance += amount
        accepted += 1
    elif kind == "withdraw" and amount > 0:
        if amount <= balance and amount <= 10000:
            balance -= amount
            accepted += 1
        else:
            rejected += 1
    else:
        rejected += 1

print("Final balance:", balance)
print("Accepted transactions:", accepted)
print("Rejected transactions:", rejected)
```

8. Test Cases

Test / Item	Expected / Result
Final balance	6500
Accepted	3
Rejected	2

9. Sample Output

```
Final balance: 6500
Accepted transactions: 3
Rejected transactions: 2
```

10. Conclusion

This ATM simulation uses tuples, a list, a loop and conditions. It is a simple example of maintaining a changing balance through transactions.

Question 5

Warehouse Inventory Reconciliation

1. Problem Statement

A small college store keeps stock of several products. The program compares opening stock, incoming stock, dispatched stock and actual closing stock. It identifies discrepancies and calculates whether more stock is needed.

2. Objectives

• Calculate expected closing stock.

• Compare expected and actual stock.

• Detect discrepancies.

• Find replenishment quantity.

3. Assumptions

• Expected stock = opening + incoming - dispatched.

• A discrepancy exists when expected and actual stock differ.

• Reorder level is 10 units.

• Replenishment is needed when actual stock is below 10.

4. Algorithm

• Start.

• Store product records.

• Calculate expected stock.

• Compare with actual stock.

• Calculate replenishment if needed.

• Display results.

• Stop.

5. Flowchart

```
START → Input / Store Data → Calculate / Process → Check Conditions → Display Result → STOP
```

6. Pseudocode

```
START
FOR each product
    expected = opening + incoming - dispatched
    difference = actual - expected
    IF difference != 0
        DISPLAY discrepancy
    END IF
    IF actual < 10
        reorder = 10 - actual
    ELSE
        reorder = 0
    END IF
END FOR
STOP
```

7. Python Source Code

```
products = [
    ["Notebooks", 50, 20, 30, 35],
    ["Pens", 40, 30, 50, 15],
    ["Files", 25, 10, 20, 12]
]

for p in products:
    name, opening, incoming, dispatched, actual = p
```

```
expected = opening + incoming - dispatched
difference = actual - expected
reorder = 10 - actual if actual < 10 else 0

print("Product:", name)
print("Expected:", expected)
print("Actual:", actual)
print("Difference:", difference)
print("Replenishment:", reorder)
print()
```

8. Test Cases

Test / Item	Expected / Result
Notebooks	Expected 40, Actual 35, Difference -5, Reorder 0
Pens	Expected 20, Actual 15, Difference -5, Reorder 0
Files	Expected 15, Actual 12, Difference -3, Reorder 0

9. Sample Output

```
Product: Notebooks
Expected: 40
Actual: 35
Difference: -5
Replenishment: 0

Product: Pens
Expected: 20
Actual: 15
Difference: -5
Replenishment: 0
```

10. Conclusion

This program shows how lists and arithmetic can be used to check stock records. It is suitable for a beginner because the logic is direct and easy to test.

Question 6

Smart Parking Fee Management

1. Problem Statement

A campus parking area charges vehicles according to parking duration and vehicle category. This program calculates the fee, handles overnight parking, and applies a simple daily maximum.

2. Objectives

• Calculate parking duration.

• Apply vehicle-based rates.

• Handle overnight parking.

• Apply a daily maximum.

• Display the final fee.

3. Assumptions

• Two-wheelers are charged 20 per hour.

• Cars are charged 40 per hour.

• Any started hour is counted as a full hour.

• Maximum daily fee is 200 for two-wheelers and 400 for cars.

• Input times are given as hours and minutes.

4. Algorithm

• Start.

• Read vehicle type and entry/exit time.

• Convert times into minutes.

• If exit time is earlier, treat it as overnight.

• Calculate hours using ceiling division.

• Multiply by hourly rate.

• Apply daily maximum.

• Display fee.

• Stop.

5. Flowchart

```
START → Input / Store Data → Calculate / Process → Check Conditions → Display Result → STOP
```

6. Pseudocode

```
START
INPUT vehicle, entry_hour, entry_minute, exit_hour, exit_minute
entry = entry_hour*60 + entry_minute
exit = exit_hour*60 + exit_minute
IF exit < entry
    exit = exit + 24*60
END IF
duration = exit - entry
hours = CEILING(duration / 60)
IF vehicle = "bike"
    rate = 20
    limit = 200
ELSE
    rate = 40
    limit = 400
END IF
fee = hours * rate
```

```
IF fee > limit
    fee = limit
END IF
DISPLAY fee
STOP
```

7. Python Source Code

```
import math

vehicle = "car"
entry = (20 * 60) + 30
exit_time = (1 * 60) + 15

if exit_time < entry:
    exit_time += 24 * 60

duration = exit_time - entry
hours = math.ceil(duration / 60)

if vehicle == "bike":
    rate = 20
    limit = 200
else:
    rate = 40
    limit = 400

fee = hours * rate
if fee > limit:
    fee = limit

print("Vehicle:", vehicle)
print("Parking hours:", hours)
print("Parking fee:", fee)
```

8. Test Cases

Test / Item	Expected / Result
Vehicle	Car
Duration	4 hours 45 minutes
Charged hours	5
Fee	200

9. Sample Output

```
Vehicle: car
Parking hours: 5
Parking fee: 200
```

10. Conclusion

This beginner-level program uses time conversion, arithmetic, if-else statements and a maximum fee rule. The example also demonstrates overnight parking.

Question 7

Multi-Day Water Tank Simulation

1. Problem Statement

A college hostel has a water tank. Every day, water enters and leaves the tank. The program updates the water level, detects overflow or shortage, and finds the first day when a target level is reached.

2. Objectives

• Update tank level day by day.

• Detect overflow.

• Detect shortage.

• Find when a target level is reached.

3. Assumptions

• Tank capacity is 1000 litres.

• Initial level is 500 litres.

• If level goes above capacity, overflow is recorded and level is set to capacity.

• If level becomes negative, shortage is recorded and level is set to zero.

• Target level is 800 litres.

4. Algorithm

• Start with initial level.

• For each day, add inflow and subtract outflow.

• Check overflow and shortage.

• Check target level.

• Display final level and events.

• Stop.

5. Flowchart

```
START → Input / Store Data → Calculate / Process → Check Conditions → Display Result → STOP
```

6. Pseudocode

```
START
level = 500
FOR each day
    level = level + inflow - outflow
    IF level > 1000
        DISPLAY "Overflow"
        level = 1000
    END IF
    IF level < 0
        DISPLAY "Shortage"
        level = 0
    END IF
    IF level >= 800 AND target_day is not found
        save day
    END IF
END FOR
DISPLAY final level and target day
STOP
```

7. Python Source Code

```
capacity = 1000
level = 500
target = 800
inflow = [400, 300, 100, 50]
```

```
outflow = [100, 150, 250, 700]
target_day = 0

for i in range(len(inflow)):
    level = level + inflow[i] - outflow[i]

    if level > capacity:
        print("Day", i + 1, ": Overflow")
        level = capacity

    if level < 0:
        print("Day", i + 1, ": Shortage")
        level = 0

    if level >= target and target_day == 0:
        target_day = i + 1

    print("Day", i + 1, "level:", level)

print("Final water level:", level)
print("First target day:", target_day)
```

8. Test Cases

Test / Item	Expected / Result
Day 1	800
Day 2	950
Day 3	800
Day 4	150
First target day	1

9. Sample Output

```
Day 1 level: 800
Day 2 level: 950
Day 3 level: 800
Day 4 level: 150
Final water level: 150
First target day: 1
```

10. Conclusion

This program uses lists and a loop to simulate changing water levels. It gives a simple introduction to real-world simulation using Python.

Question 8

Student Performance Trend Analysis

1. Problem Statement

A first-year class wants to compare marks from different examinations. The program calculates improvement or decline, identifies weak subjects, and gives a simple performance classification.

2. Objectives

• Compare two examination scores.

• Calculate improvement or decline.

• Identify weak subjects.

• Classify overall performance.

3. Assumptions

• Marks are out of 100.

• A subject is weak when the latest mark is below 40.

• Improvement = latest mark - previous mark.

• Average of latest marks is used for classification.

• Average 75 or more = Excellent; 60 or more = Good; otherwise Needs Improvement.

4. Algorithm

• Start.

• Store subjects and two sets of marks.

• Find change for every subject.

• Check weak subjects.

• Calculate latest average.

• Classify performance.

• Display report.

• Stop.

5. Flowchart

```
START → Input / Store Data → Calculate / Process → Check Conditions → Display Result → STOP
```

6. Pseudocode

```
START
FOR each subject
    change = latest - previous
    IF latest < 40
        DISPLAY "Weak"
    END IF
END FOR
average = sum(latest) / number of subjects
IF average >= 75
    class = "Excellent"
ELSE IF average >= 60
    class = "Good"
ELSE
    class = "Needs Improvement"
END IF
DISPLAY report
STOP
```

7. Python Source Code

```
subjects = ["Python", "Maths", "Web", "English"]
```

```
previous = [60, 70, 55, 80]
latest = [72, 65, 48, 85]

total = 0
for i in range(len(subjects)):
    change = latest[i] - previous[i]
    print(subjects[i], "Change:", change)

    if latest[i] < 40:
        print("Weak subject")

    total += latest[i]

average = total / len(latest)

if average >= 75:
    result = "Excellent"
elif average >= 60:
    result = "Good"
else:
    result = "Needs Improvement"

print("Latest average:", round(average, 2))
print("Performance:", result)
```

8. Test Cases

Test / Item	Expected / Result
Python	+12
Maths	-5
Web	-7
English	+5
Average	67.5
Classification	Good

9. Sample Output

```
Python Change: 12
Maths Change: -5
Web Change: -7
English Change: 5
Latest average: 67.5
Performance: Good
```

10. Conclusion

This program demonstrates lists, loops, arithmetic and conditions. It gives a simple way for a first-year student to understand how performance trends can be measured.

Question 9

Fuel Consumption and Route Comparison

1. Problem Statement

A student wants to compare travel options for a college trip. The program calculates fuel needed and travel cost for different vehicles and chooses the lowest-cost feasible option after considering a reserve requirement.

2. Objectives

• Calculate fuel required.

• Calculate trip cost.

• Check reserve requirement.

• Compare vehicles.

• Select the lowest-cost feasible option.

3. Assumptions

• Fuel price is 105 per litre.

• Required reserve is 10% of trip fuel.

• Mileage is km per litre.

• A vehicle is feasible when available fuel is enough for trip fuel plus reserve.

• No advanced libraries are used.

4. Algorithm

• Start.

• Store vehicle data.

• Calculate fuel needed = distance / mileage.

• Calculate reserve.

• Check available fuel.

• Calculate cost for feasible vehicles.

• Select minimum cost.

• Display best vehicle.

• Stop.

5. Flowchart

```
START → Input / Store Data → Calculate / Process → Check Conditions → Display Result → STOP
```

6. Pseudocode

```
START
distance = 180
FOR each vehicle
    fuel = distance / mileage
    reserve = fuel * 0.10
    required = fuel + reserve
    IF available >= required
        cost = fuel * fuel_price
        compare cost
    END IF
END FOR
DISPLAY lowest-cost feasible vehicle
STOP
```

7. Python Source Code

```
distance = 180
```

```
fuel_price = 105

vehicles = [
    ["Bike", 45, 5],
    ["Car", 15, 20],
    ["Van", 10, 25]
]

best_name = ""
best_cost = 999999

for name, mileage, available in vehicles:
    fuel = distance / mileage
    reserve = fuel * 0.10
    required = fuel + reserve

    if available >= required:
        cost = fuel * fuel_price
        print(name, "Fuel:", round(fuel, 2), "Cost:", round(cost, 2))
        if cost < best_cost:
            best_cost = cost
            best_name = name

print("Lowest-cost feasible vehicle:", best_name)
print("Cost:", round(best_cost, 2))
```

8. Test Cases

Test / Item	Expected / Result
Bike	Fuel 4.00 L, Cost 420.00, Feasible
Car	Fuel 12.00 L, Cost 1260.00, Feasible
Van	Fuel 18.00 L, Cost 1890.00, Feasible
Best	Bike

9. Sample Output

```
Bike Fuel: 4.0 Cost: 420.0
Car Fuel: 12.0 Cost: 1260.0
Van Fuel: 18.0 Cost: 1890.0
Lowest-cost feasible vehicle: Bike
Cost: 420.0
```

10. Conclusion

This program compares several travel choices using simple arithmetic and conditions. It is a beginner-friendly example of selecting the lowest-cost feasible option.

Question 10

Multi-Product Shopping Budget Constraint

1. Problem Statement

A student has a fixed budget for buying college supplies. The program calculates the basket cost, applies a simple discount, and identifies how many units of selected items must be reduced when the basket exceeds the budget.

2. Objectives

• Calculate total shopping cost.

• Apply discount condition.

• Compare final cost with budget.

• Reduce quantities when necessary.

• Display final basket cost.

3. Assumptions

• Budget is 3000.

• A 10% discount is given when subtotal is at least 2500.

• If the discounted total exceeds budget, reduce the quantity of the most expensive item one unit at a time.

• Quantities cannot become negative.

4. Algorithm

• Start.

• Store product prices and quantities.

• Calculate subtotal.

• Apply discount if eligible.

• While total exceeds budget, reduce expensive item quantity.

• Display final total and quantities.

• Stop.

5. Flowchart

```
START → Input / Store Data → Calculate / Process → Check Conditions → Display Result → STOP
```

6. Pseudocode

```
START
INPUT products, quantities, prices
subtotal = sum(price * quantity)
IF subtotal >= 2500
    total = subtotal * 0.90
ELSE
    total = subtotal
END IF
WHILE total > budget AND quantity of expensive item > 0
    reduce quantity by 1
    recalculate total
END WHILE
DISPLAY basket
STOP
```

7. Python Source Code

```
products = ["Notebook", "Backpack", "Calculator"]
prices = [80, 1200, 900]
qty = [5, 2, 1]
budget = 3000

subtotal = 0
```

```
for i in range(len(products)):
    subtotal += prices[i] * qty[i]

if subtotal >= 2500:
    total = subtotal * 0.90
else:
    total = subtotal

while total > budget and qty[1] > 0:
    qty[1] -= 1
    subtotal = 0
    for i in range(len(products)):
        subtotal += prices[i] * qty[i]
    total = subtotal * 0.90 if subtotal >= 2500 else subtotal

print("Final basket:")
for i in range(len(products)):
    print(products[i], ":", qty[i])

print("Final cost:", round(total, 2))
print("Budget:", budget)
```

8. Test Cases

Test / Item	Expected / Result
Notebook	5
Backpack	1
Calculator	1
Final cost	1980.00
Budget	3000

9. Sample Output

```
Final basket:
Notebook : 5
Backpack : 1
Calculator : 1
Final cost: 1980.0
Budget: 3000
```

10. Conclusion

This beginner-level program uses lists, loops, arithmetic and conditions to solve a practical budget problem without using advanced Python features.